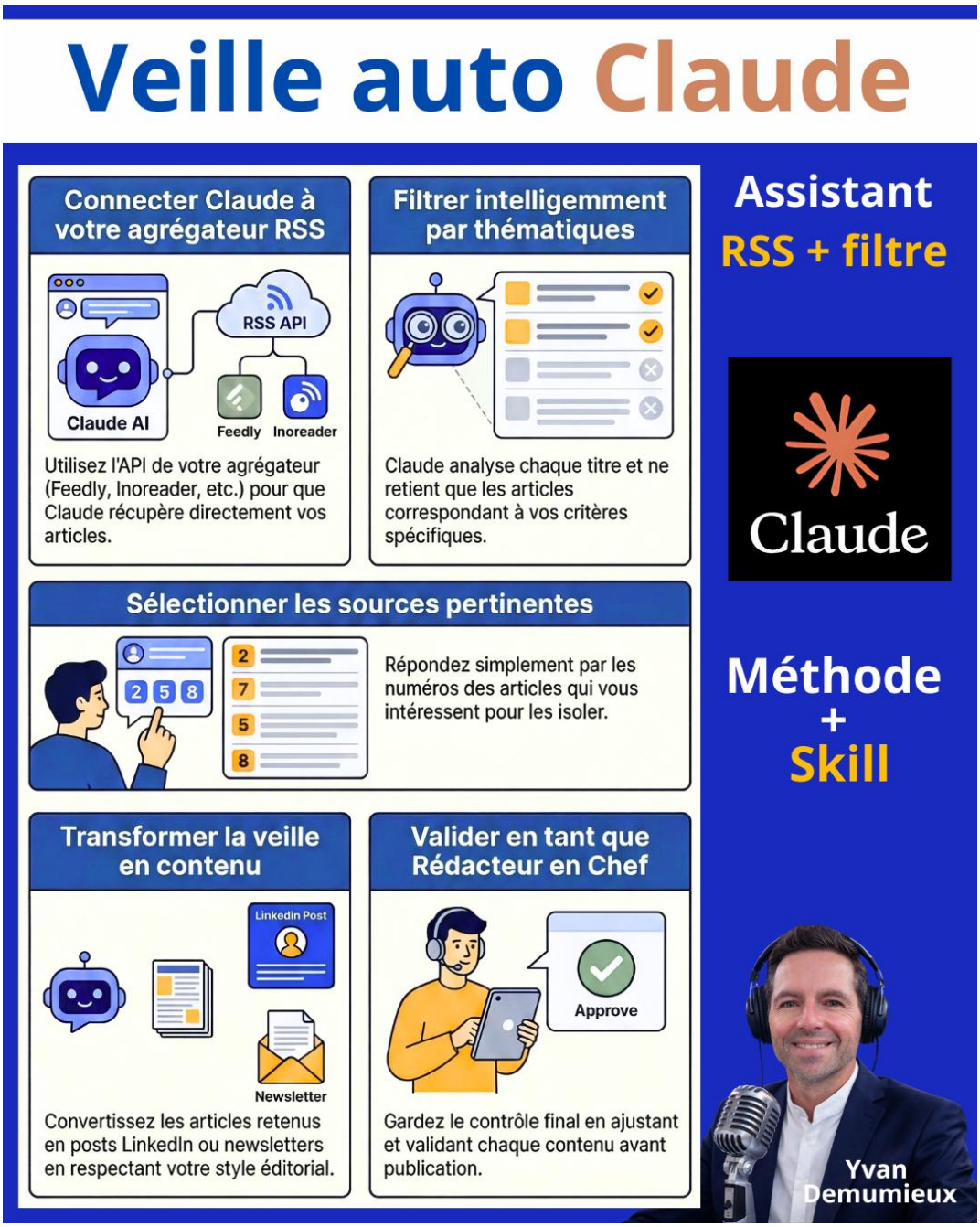
Veille IA automatique avec Claude Code et un agrégateur RSS

Construire son système de filtrage en une session — sans Make, sans n8n

Par Yvan Demumieux — AppyLearny



Pourquoi ce guide ?

La veille IA est indispensable. Mais la réalité c'est que la plupart des professionnels abandonnent au bout de quelques semaines. Pas par manque de motivation — par noyade.

50, 80, 100 articles par jour dans un agrégateur RSS. Une boîte email débordante. On ne sait plus quoi lire, on reporte, et finalement on ne lit plus rien.

Les solutions classiques — automatiser avec Make ou n8n — demandent du temps, des compétences techniques et un abonnement supplémentaire. Beaucoup repartent sans avoir rien mis en place.

Ce guide montre une alternative : construire un système de veille intelligent en une session de travail, avec Claude Code et n'importe quel agrégateur RSS. Claude écrit les scripts, connecte les APIs, et filtre les articles selon tes critères. Tu valides. Tu gardes le contrôle éditorial.

Ce dont tu as besoin

Un agrégateur RSS (au choix)

Un agrégateur RSS collecte automatiquement les articles de tes sources préférées (blogs, newsletters, sites d'actu) et les centralise en un seul endroit.

Options gratuites et open source :

- **FreshRSS** — auto-hébergé, gratuit, complet
- **Miniflux** — minimaliste, rapide, auto-hébergeable
- **Karakeep** — auto-hébergé, aussi gestionnaire de bookmarks

Options payantes accessibles :

- **Feedly** — interface soignée, à partir de 8$/mois
- **Inoreader** — puissant, nombreux filtres natifs, à partir de 5$/mois
- **NewsBlur** — open source avec option cloud

Le critère essentiel : que ton agrégateur dispose d'une **API accessible**. La plupart en ont une, souvent documentée dans leurs paramètres.

Claude Code

L'interface en ligne de commande de Claude (Anthropic). Elle permet d'avoir Claude comme assistant dans ton terminal, capable de créer des fichiers, exécuter des scripts et appeler des APIs.

Installation : disponible sur le site d'Anthropic.

Python

Un langage de programmation simple, déjà installé sur la plupart des machines. Si ce n'est pas le cas : python.org, installation en 3 minutes.

Tu n'as pas besoin de savoir programmer. Claude s'en occupe.

Comment fonctionne le système

Le pipeline complet tient en 4 étapes :

Flux RSS → Agrégateur → Claude analyse → Ta décision → Liste sélection

Étape 1 — Collecte automatique Ton agrégateur reçoit les articles de tous tes flux RSS en continu. Tu n'as rien à faire.

Étape 2 — Récupération par Claude Un script Python interroge l'API de ton agrégateur et récupère les articles récents. Claude filtre d'emblée les sources parasites connues (anciens flux, domaines hors sujet).

Étape 3 — Analyse et présentation Claude lit chaque titre et description, applique tes critères de filtrage, et te présente une liste numérotée :

"J'ai récupéré 50 articles, voici 12 pertinents :" 1. [HAUTE] Gemini 3 Deep Think — nouveau modèle majeur Google 2. [HAUTE] Import AI 448 — agent Bytedance qui écrit du code 3. [HAUTE] Notre 2026 Responsible AI Report — gouvernance IA ...

Étape 4 — Ta décision Claude pose la question : > "Qu'est-ce que tu veux récupérer ? Réponds : tout, rien, ou les numéros (ex: 1 3 7)"

Tu réponds en quelques secondes. Claude pousse uniquement tes sélections dans une liste dédiée de ton agrégateur.

C'est cette étape qui fait toute la différence : **Claude ne décide pas à ta place**. Il réduit le bruit, tu gardes le contrôle éditorial.

Construire le système avec Claude Code

Session 1 — Créer le script de récupération (fetch)

Ouvre Claude Code dans ton dossier de travail et décris ce que tu veux :

"Je veux un script Python qui récupère les 50 derniers articles de mon agrégateur RSS via son API et me retourne un JSON avec index, titre, URL et description pour chaque article."

Claude va te demander :

- L'URL de base de l'API de ton agrégateur
- Ta clé API (disponible dans les paramètres de ton agrégateur)
- Le format de réponse attendu

Il écrit le script. Tu le testes. Si quelque chose ne va pas, tu décris le problème en langage naturel et Claude corrige.

Exemple de commande finale :

python scripts/veille_fetch.py

Le script retourne un JSON des articles récents, prêt à être analysé.

Session 1 (suite) — Définir tes critères de filtrage

C'est l'étape la plus importante. Tu décris à Claude les thématiques qui t'intéressent et celles à ignorer.

Exemple de critères (à adapter à ton contexte) :

Toujours retenir : - Gouvernance IA, réglementation, AI Act, conformité - Agents IA, automatisation, MCP, multi-agent, workflows - Nouveautés sur mes outils : Claude, ChatGPT, Gemini, Perplexity, NotebookLM

Toujours ignorer : - Consumer tech, gadgets, deals - Politique, sport, divertissement - Levées de fonds sans impact produit - Tutoriels génériques sans nouveauté

Ces critères sont enregistrés dans un fichier de configuration que Claude consulte à chaque session.

Session 1 (suite) — Créer le script d'envoi (push)

Tu décris ce que tu veux faire avec les articles retenus :

“Je veux un script Python qui prend une liste d’articles en JSON et les crée dans une liste spécifique de mon agrégateur via l’API, avec les tags ‘veille’ et ‘selection’, et la date dans la note.”

Claude écrit le script, gère les appels API, les doublons et la gestion d’erreurs.

Note technique importante pour Karakeep : la création d’un bookmark dans une liste spécifique nécessite deux appels API — d’abord créer le bookmark, puis l’assigner à la liste. Claude le découvre et l’adapte automatiquement lors des tests.

Créer la commande /veille

Une fois les deux scripts fonctionnels, Claude crée une commande slash dans Claude Code :

/veille

Cette commande orchestre tout le pipeline en une seule invocation : fetch → analyse → présentation → décision → push.

Choisir ses sources RSS

La qualité de ta veille dépend de tes sources. Voici une sélection selon les thématiques IA professionnelles :

Agents IA et recherche : - Import AI (Jack Clark) — importai.substack.com - The Gradient — thegradient.pub - Ahead of AI — magazine.sebastianraschka.com

Actualité IA générale : - Ben’s Bites — bensbites.co - The Rundown AI — therundown.ai - TLDR AI — tldr.tech/ai

Outils et productivité IA : - Lenny’s Newsletter — lennysnewsletter.com - The Neuron — theneurondaily.com

Gouvernance et régulation : - MIT Technology Review AI — technologyreview.com - Future of Life Institute — futureoflife.org

Conseil : commence avec 5 à 8 sources maximum. Mieux vaut peu de sources de qualité que beaucoup de bruit.

Et après : recycler les articles retenus

La veille ne s'arrête pas à la sélection. Un article retenu est une matière première.

Depuis le même environnement Claude Code, tu peux transformer n'importe quelle source retenue en :

- **Post LinkedIn** — en appliquant tes prompts de style personnalisés
- **Angle de newsletter** — reformulé dans ton ton éditorial
- **Contenu pédagogique** — adapté pour une formation ou un atelier

Claude connaît ton style, ton ton, tes formats. Tu relis, ajustes, valides. Semi-automatique — la décision éditoriale reste la tienne.

Veille → sélection → création de contenu
Tout dans le même outil, la même session.

Ce que ce système ne fait pas

Pour éviter les fausses attentes :

- Il ne lit pas le contenu complet des articles (seulement titres et descriptions)
- Il ne remplace pas une lecture attentive sur les sujets importants
- Il ne gère pas les paywalls ou les contenus privés
- Il nécessite une session Claude Code active (pas un démon qui tourne en arrière-plan)

Pour une veille entièrement automatisée sans intervention humaine, des outils comme n8n ou Make restent pertinents. Ce système est fait pour ceux qui veulent garder la main tout en éliminant 80% du bruit.

En résumé

	Avant	Avec ce système
Temps de tri	20-30 min/jour	2-3 min/session
Compétences requises	Make/n8n, webhooks, APIs	Décrire en français
Coût supplémentaire	Abonnement automation	0 (inclus dans Claude)
Contrôle éditorial	Total (mais épuisant)	Total (et fluide)
Mise en place	1 weekend	1 session